

# ReconForge Configuration Guide

Version 1.0 — Last updated: 2026-03-21

## Overview

ReconForge uses two YAML configuration files as the single source of truth:

| File                 | Purpose                                                                         |
|----------------------|---------------------------------------------------------------------------------|
| config/tools.yaml    | Tool binary paths, default arguments, timeouts, scan profiles, safety settings  |
| config/profiles.yaml | OPSEC-aware scan profiles with timing, technique toggles, and noise-level gates |

There are no fallback namespaces, no environment variable overrides, and no hardcoded config overrides. The YAML files are authoritative.

## tools.yaml

### Structure

Each tool entry lives under the top-level `tools:` key:

```
tools:
  nmap:
    binary: nmap
    description: "Network mapper - host discovery, port scanning, service enumeration"
    required: true
    default_timeout: 600
    install_cmd: "sudo apt install -y nmap"
    scan_profiles:
      ping_sweep:
        args: "-sn"
        timeout: 120
        detection: low
      syn_scan:
        args: "-sS --open"
        timeout: 600
        detection: medium
        requires_root: true
      version_scan:
        args: "-sV --version-intensity 5"
        timeout: 600
        detection: medium
```

## Top-Level Tool Fields

| Field                        | Type   | Description                                                       |
|------------------------------|--------|-------------------------------------------------------------------|
| <code>binary</code>          | string | Primary binary name (e.g., <code>nmap</code> )                    |
| <code>alt_binary</code>      | string | Alternative binary name (e.g., <code>impacket-GetADUsers</code> ) |
| <code>description</code>     | string | Human-readable tool description                                   |
| <code>required</code>        | bool   | Whether the tool is required for the module                       |
| <code>default_timeout</code> | int    | Default timeout in seconds                                        |
| <code>install_cmd</code>     | string | Installation command                                              |
| <code>detection</code>       | string | Top-level detection/noise level                                   |
| <code>opt_in_only</code>     | bool   | Requires explicit user opt-in (e.g., <code>hydra</code> )         |
| <code>warning</code>         | string | Safety warning displayed when tool is used                        |

## Mode / Scan Profile Fields

Tools define either `modes:` or `scan_profiles:` (both are treated identically by `ToolConfig` ):

| Field                      | Type   | Description                                                                                      |
|----------------------------|--------|--------------------------------------------------------------------------------------------------|
| <code>args</code>          | string | Raw argument string for this mode                                                                |
| <code>timeout</code>       | int    | Mode-specific timeout override                                                                   |
| <code>detection</code>     | string | Noise level: <code>low</code> , <code>medium</code> , <code>high</code> , <code>very_high</code> |
| <code>requires_root</code> | bool   | Whether the mode requires root privileges                                                        |

## Safety Block (hydra example)

```
hydra:
  binary: hydra
  opt_in_only: true
  warning: "Brute-force testing can trigger lockouts and IDS alerts"
  safety:
    max_tasks: 4
    wait_time: 3
    max_attempts_per_account: 10
```

## Collection Methods (bloodhound example)

```
bloodhound_python:
  binary: bloodhound-python
  collection_methods:
    all:
      args: "-c All"
      timeout: 900
    default:
      args: "-c DCOnly"
      timeout: 300
    stealth:
      args: "-c DCOnly --dns-tcp"
      timeout: 300
```

## Currently Configured Tools

**Network Module:** nmap, enum4linux, smbclient, ldapsearch, hydra

**AD Module:** enum4linux\_ng, impacket\_getadusers, impacket\_getnpusers, impacket\_lookupsid, impacket\_rpcdump, nmap\_ad, bloodhound\_python, netexec, impacket\_finddelegation, impacket\_getuserspns

**Web Module:** whatweb, wafw00f, nikto, gobuster, ffuf, wpscan, nuclei, sqlmap, testssl

**API Module:** httpx, arjun, ffuf\_api, nuclei\_api

**Surface Module:** nmap\_surface, httpx\_surface

## profiles.yaml

### Structure

Each profile lives under the top-level `profiles:` key:

```

profiles:
  stealth:
    description: "Minimal noise - avoid detection"
    opsec_mode: stealth
    timing:
      nmap_timing: T2
      scan_delay: "500ms"
      max_retries: 1
    scanning:
      port_range: "21,22,23,25,53,80,88,..."
      scan_type: syn
      version_detection: false
    enumeration:
      enum4linux: false
      smb_scripts: false
    authentication:
      brute_force: false
      anonymous_checks: true
    allowed_noise_levels:
      - low

```

## Base Profiles

| Profile    | OPSEC Mode | Noise Levels                 | Description                            |
|------------|------------|------------------------------|----------------------------------------|
| stealth    | stealth    | low                          | Minimal noise — avoid detection        |
| normal     | normal     | low, medium                  | Balanced — standard engagement         |
| aggressive | aggressive | low, medium, high, very_high | Maximum coverage — CTF/Lab             |
| ctf        | aggressive | low, medium, high, very_high | CTF optimized — fast and comprehensive |

## Module-Specific Profiles

Each module has dedicated profile variants that override the base profile with module-specific settings:

**AD:** stealth\_ad, normal\_ad, aggressive\_ad

**Surface:** stealth\_surface, normal\_surface, aggressive\_surface

**API:** stealth\_api, normal\_api, aggressive\_api

**Web:** stealth\_web, normal\_web, aggressive\_web

Example — stealth\_ad restricts phases and disables noisy techniques:

```

stealth_ad:
  description: "AD recon - minimal noise, anonymous/passive only"
  opsec_mode: stealth
  timing:
    nmap_timing: T2
    scan_delay: "500ms"
  ad:
    phases:
      - passive
    anonymous_only: true
    ldap_queries: true
    smb_null_session: true
    rid_cycling: false
    enum4linux_ng: false
    impacket: false
    allowed_noise_levels:
      - low

```

## Profile Resolution Order

When `ProfileLoader` is initialized with `(config, opsec_mode, module)`, it resolves:

1. **Module-specific variant** (most specific): `{base_mode}_{module}` (e.g., `stealth_ad`)
2. **Exact match**: `opsec_mode` as-is
3. **Base mode**: via canonical mapping (`stealth`, `normal`, `aggressive`)
4. **Empty dict**: all defaults

## ToolConfig Accessor

The `ToolConfig` class provides typed access to tool YAML configuration inside tool wrappers:

```

from core.tool_config import ToolConfig

class GobusterTool:
    def __init__(self, runner, logger, output_dir, opsec_mode="normal", config=None):
        self.tool_cfg = ToolConfig(config, "gobuster")

    def dir_scan(self, target, timeout=600):
        # Timeout resolution: mode-specific → tool default → caller default
        effective_timeout = self.tool_cfg.mode_timeout("dir", timeout)
        threads = self.tool_cfg.mode_value("dir", "threads", 50)
        args = self.tool_cfg.mode_args("dir", default="dir -t 50")

```

## Timeout Resolution Hierarchy

mode-specific timeout  tool default\_timeout  caller default

`ToolConfig.effective_timeout(mode, caller_default)` resolves this chain automatically.

## Backward Compatibility

`ToolConfig(None, "tool")` returns caller-supplied defaults for every accessor — providing full backward compatibility for tools not yet consuming configuration.

## Key Methods

| Method                                               | Description                               |
|------------------------------------------------------|-------------------------------------------|
| <code>binary</code>                                  | Primary binary name                       |
| <code>alt_binary</code>                              | Alternative binary name                   |
| <code>required</code>                                | Whether tool is required                  |
| <code>default_timeout</code>                         | Top-level timeout from YAML               |
| <code>has_config</code>                              | Whether a non-empty YAML entry was loaded |
| <code>mode_timeout(mode, default)</code>             | Timeout for a named mode                  |
| <code>mode_args(mode, default)</code>                | Argument string for a mode                |
| <code>mode_detection(mode, default)</code>           | Detection level for a mode                |
| <code>mode_value(mode, key, default)</code>          | Arbitrary key from a mode entry           |
| <code>mode_requires_root(mode)</code>                | Whether a scan profile requires root      |
| <code>safety(key, default)</code>                    | Read from the <code>safety: block</code>  |
| <code>collection(method, key, default)</code>        | Read from <code>collection_methods</code> |
| <code>get(dotted_key, default)</code>                | Dot-notation generic getter               |
| <code>effective_timeout(mode, caller_default)</code> | Unified timeout resolution                |

## ProfileLoader

The `ProfileLoader` class resolves and queries OPSEC profiles:

```

from core.profile_loader import ProfileLoader

loader = ProfileLoader(config_loader, opsec_mode="stealth", module="ad")

# Access timing
timing = loader.timing          # {"nmap_timing": "T2", "scan_delay":
"500ms", ...}
nmap_t = loader.nmap_timing    # "T2"

# Check noise levels
allowed = loader.allowed_noise # ["low"]

# Check technique toggles
can_rid = loader.is_technique_enabled("rid_cycling") # False for stealth_ad

# Get enabled phases
phases = loader.enabled_phases() # ["passive"] for stealth_ad

# Deep access
anon = loader.get("ad.anonymous_only", default=False) # True for stealth_ad

```

## Key Properties

| Property / Method                            | Description                                         |
|----------------------------------------------|-----------------------------------------------------|
| <code>profile_data</code>                    | Full resolved profile dict                          |
| <code>opsec_mode</code>                      | Effective OPSEC mode string                         |
| <code>timing</code>                          | Timing configuration dict                           |
| <code>allowed_noise</code>                   | List of allowed noise levels                        |
| <code>nmap_timing</code>                     | nmap <code>-T</code> value (e.g., <code>T2</code> ) |
| <code>scan_delay</code>                      | nmap <code>--scan-delay</code> value                |
| <code>max_retries</code>                     | nmap <code>--max-retries</code> value               |
| <code>section(key)</code>                    | Top-level section of the profile                    |
| <code>get(dotted_key, default)</code>        | Deep dot-notation access                            |
| <code>is_technique_enabled(technique)</code> | Check technique toggle                              |
| <code>enabled_phases()</code>                | Profile-restricted phase list (or None)             |